

NutriPrecio

Informe Sprint I

Curso: Taller de Metodos de Innovacion Agiles

Proyecto: NutriPrecio - Plataforma de comparacion de precios

Sprint: Sprint I

Fecha: Mayo 2026

Equipo de Desarrollo

Sebastian Herrera - Scrum Master

Ignacio Herrera - Product Owner

Vicente Sepulveda - Developer

Matias Ramirez - Developer

Benjamin Buzeta - Developer

Fernando Sepulveda - Developer

1. Introduccion

El presente informe documenta el desarrollo y los resultados obtenidos durante el Sprint I del proyecto NutriPrecio, una plataforma web de comparacion de precios de alimentos saludables orientada al mercado chileno. El sistema permite a los consumidores buscar, comparar y encontrar los mejores precios en supermercados y tiendas locales de Chile.

1.1 Objetivo del Sprint I

El objetivo principal del Sprint I fue implementar las funcionalidades base de la plataforma NutriPrecio, centradas en tres historias de usuario fundamentales:

- MV-03: Registro de tienda para vendedores independientes.
- MV-52: Sistema de login y registro con autenticacion por tokens.
- MV-54: Panel de control privado para vendedores.

1.2 Alcance del Incremento

El alcance del Sprint I comprendio el desarrollo del sistema de autenticacion completo (backend y frontend), el modelo de datos para tiendas con sus endpoints API, el formulario de registro de tienda en el frontend, y la vista inicial del dashboard del vendedor. Se planificaron 9 tareas con un total de 47 horas estimadas, distribuidas entre los 4 desarrolladores del equipo.

2. Metodologia y Enfoque Tecnico

2.1 Framework Scrum

El equipo adopto el framework Scrum para la gestion del Sprint I. La planificacion se realizo mediante la seleccion de items del Product Backlog priorizados por el Product Owner, los cuales se desglosaron en tareas tecnicas asignadas a cada miembro del equipo.

2.2 Herramientas Utilizadas

Backend: Django 6.0 con Django REST Framework, SQLite, django-cors-headers, Token Authentication.

Frontend: Angular 19 con componentes standalone, Angular Material, SCSS, RxJS.

Control de versiones: Git con ramas por feature y commits en modo imperativo.

Colaboracion: Plantilla de Sprint Backlog en Excel para seguimiento de tareas.

2.3 Roles del Equipo

Rol	Nombre
Scrum Master	Sebastian Herrera
Product Owner	Ignacio Herrera
Developer	Vicente Sepulveda
Developer	Matias Ramirez
Developer	Benjamin Buzeta
Developer	Fernando Sepulveda

3. Sprint Backlog

A continuacion se presenta el detalle de las 9 tareas planificadas para el Sprint I:

ID	Tarea	Responsable	Hrs Est.	Estado
MV-03	Backend: Crear tabla en BDD para Perfiles de Tienda, v	Vicente Sepulveda	4	Hecho
MV-03	Frontend: Crear formulario para que el vendedor ingrese	Benjamin Buzeta	5	Hecho
MV-03	Backend: Crear endpoint para recibir y guardar la infor...	Matias Ramirez	5	Hecho
MV-52	Backend: Configurar la base de datos para usuarios, y e	Vicente Sepulveda	4	Hecho
MV-52	Frontend: Disenar interfaz del formulario de Registro e...	Fernando Sepulveda	5	Hecho
MV-52	Backend: Desarrollar logica para validacion de credenci	Benjamin Buzeta	5	Hecho
MV-54	Backend: Configurar rutas protegidas en el sistema, par	Vicente Sepulveda	5	Hecho
MV-54	Frontend: Disenar y maquetar el Layout del Dashboard.	Benjamin Buzeta	8	Hecho parcialmente
MV-54	Frontend: Crear vista de Inicio del Dashboard, mostrand	Sebastian Herrera	6	Hecho parcialmente
	TOTAL		47	7 Hecho, 2 Parcial

Resumen: 7 de 9 tareas completadas (78%), 2 tareas parciales (22%). Horas estimadas: 47. Horas consumidas: 21.

4. Diagrama de Casos de Uso

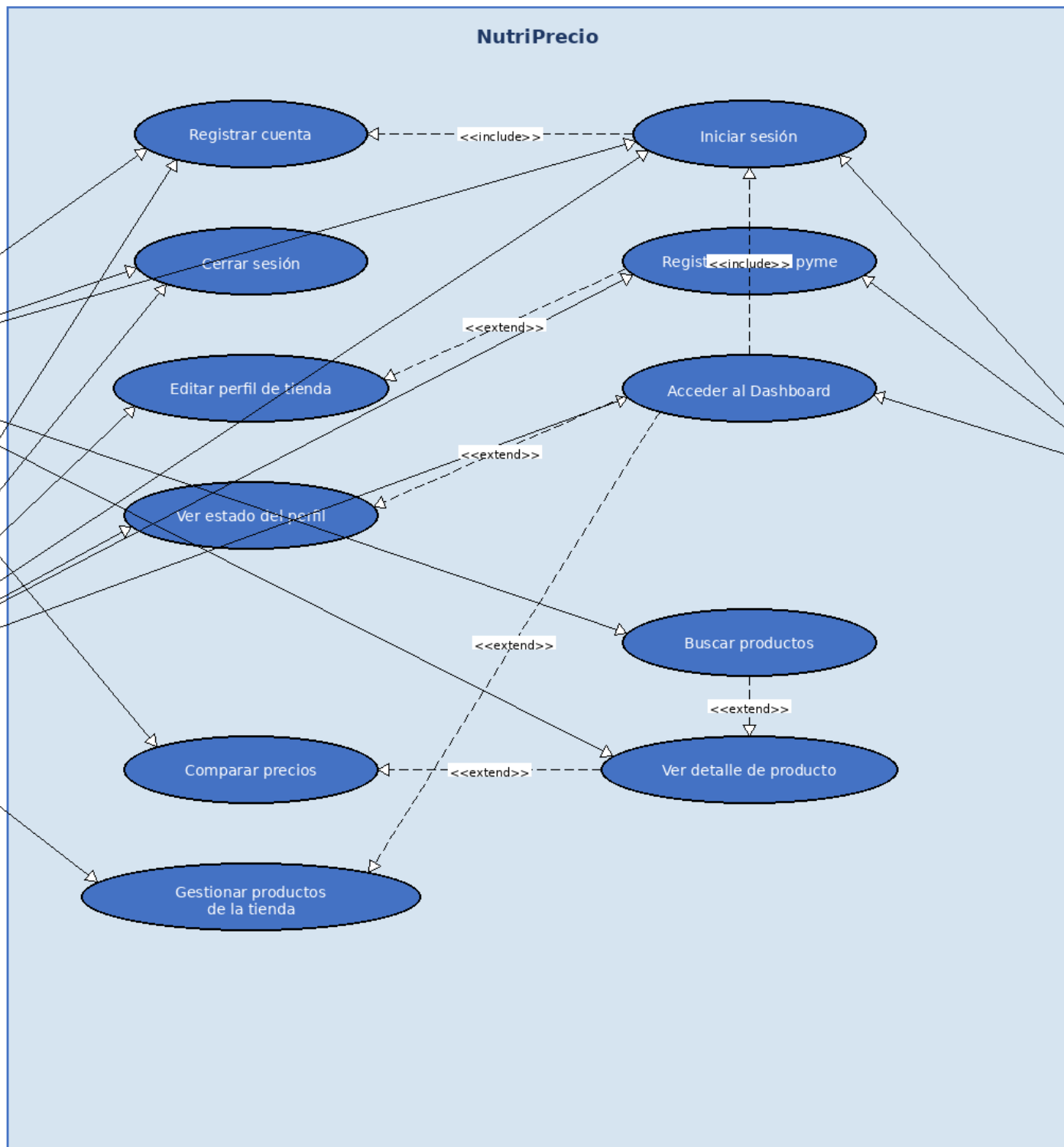


Figura 1: Diagrama de Casos de Uso del sistema NutriPrecio

4.1 Actores del Sistema

Usuario No Autenticado: Navega, busca productos y compara precios sin registro.

Usuario Registrado (Comprador): Crea cuenta, inicia sesion, accede a funcionalidades personalizadas.

Vendedor (Seller): Registra su tienda, gestiona su perfil, accede al dashboard privado.

Sistema de Autenticacion: Gestiona registro, validacion de credenciales y tokens de sesion.

4.2 Casos de Uso Principales

- Registrarse
- Iniciar Sesion
- Buscar Productos
- Comparar Precios
- Registrar Tienda
- Acceder al Dashboard

5. Diagrama de Clases

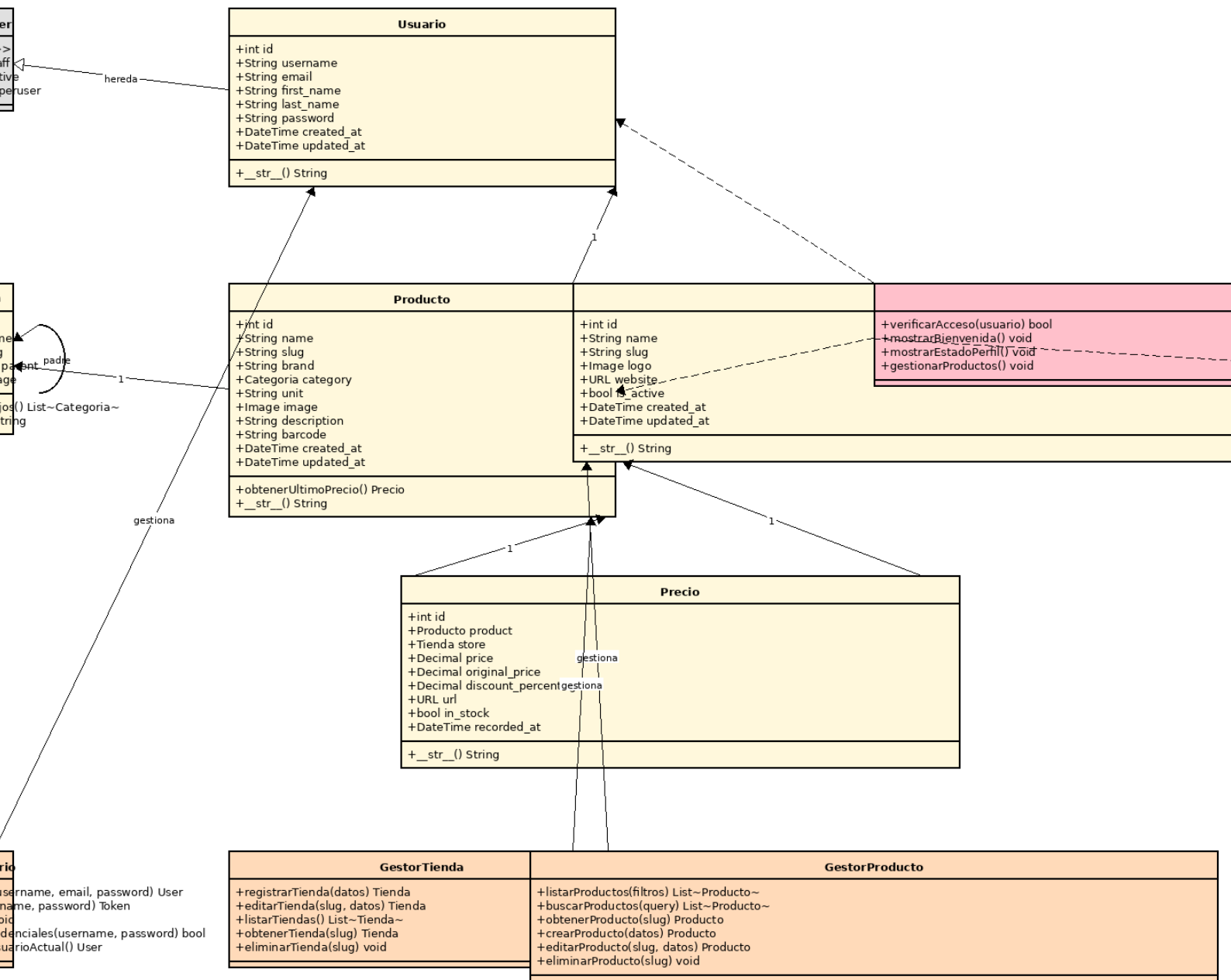


Figura 2: Diagrama de Clases del sistema NutriPrecio

5.1 Estructura del Sistema

El diagrama de clases muestra la arquitectura del sistema NutriPrecio, organizada en capas que separan la interfaz de usuario, la logica de negocio y el acceso a datos.

5.2 Clases Principales

User: Extiende AbstractUser. Email unico, relacion con Store via owner.

Store: Tienda/pyme con nombre, slug, logo, sitio web, owner (FK a User).

Product: Catalogo con nombre, marca, categoria, unidad, imagen, descripcion.

Category: Categorias jerarquicas con referencia padre-hijo.

Price: Precio vinculado a Product y Store. Incluye descuento y disponibilidad.

AuthService: Servicio Angular para login, registro, logout y gestion de token.

ApiService: Servicio Angular base para comunicaciones HTTP tipadas.

AuthGuard: Guard de rutas que verifica autenticacion y rol de vendedor.

5.3 Relaciones

- User (1) -- (N) Store: Un vendedor puede tener una o mas tiendas.
- Category (1) -- (N) Product: Una categoria contiene multiples productos.
- Product (1) -- (N) Price: Un producto tiene multiples registros de precio.
- Store (1) -- (N) Price: Una tienda tiene multiples registros de precio.

6. Diagrama de Actividades

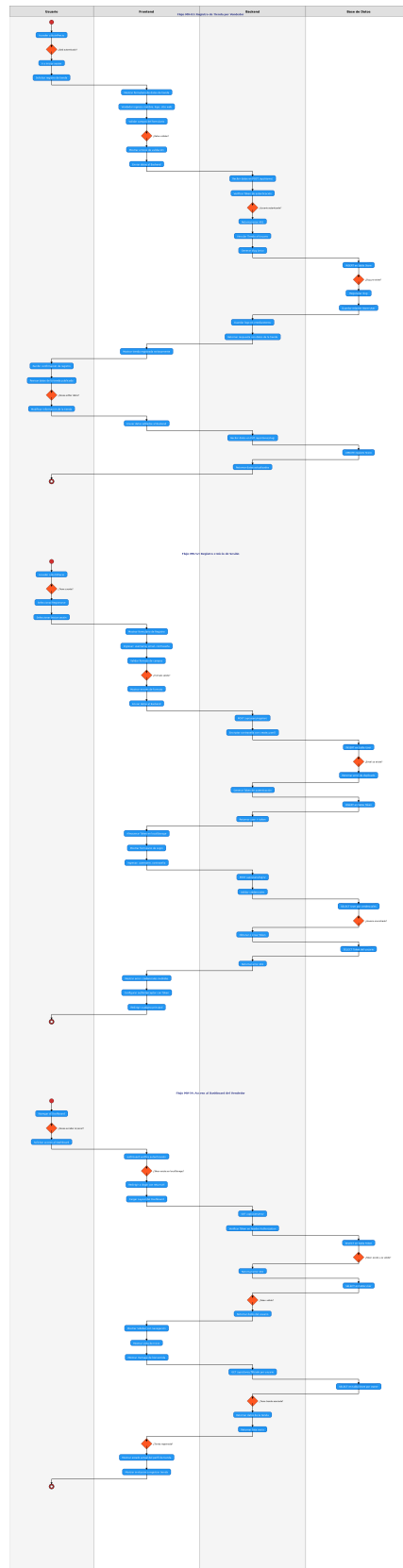


Figura 3: Diagrama de Actividades con los 3 flujos principales

6.1 Flujo MV-52: Login y Registro

El usuario accede al formulario de login o registro. El sistema valida los campos, envia la solicitud al backend, que verifica credenciales contra la base de datos. Si son validas, genera un token de autenticacion y lo retorna al frontend. En registro, se crea el usuario con contraseña encriptada mediante `create_user()` de Django.

6.2 Flujo MV-03: Registro de Tienda

El vendedor autenticado accede al formulario desde el dashboard. El frontend construye `FormData` con los datos de la tienda y lo envia a `POST /api/stores/`. El backend valida el rol de vendedor, crea la instancia de `Store` vinculada al owner, y retorna la tienda creada.

6.3 Flujo MV-54: Dashboard del Vendedor

Al acceder a `/dashboard`, el `authGuard` verifica autenticacion y rol `is_seller`. Si ambas condiciones se cumplen, se carga la vista con mensaje de bienvenida personalizado, perfil del usuario con avatar, indicadores de estado, y acciones rapidas. Si no es vendedor, es redirigido al home.

7. Sprint Review

7.1 Funcionalidades Entregadas

MV-52: Login y Registro (COMPLETO)

Registro con encriptacion de contraseñas. Tokens de autenticacion. Interceptor `HTTP authInterceptor`. Guard `authGuard` con validacion de token y rol. Formularios con validacion.

MV-03: Registro de Tienda (COMPLETO)

Modelo `Store` con owner FK. Serializer y `ViewSet` con permisos. Endpoint CRUD en `/api/stores/`. Formulario frontend `store-form.component` integrado al dashboard.

MV-54: Dashboard del Vendedor (COMPLETO)

Mensaje de bienvenida personalizado. Perfil con avatar de iniciales. Indicadores de estado. Acciones rapidas con navegacion. Validacion de rol vendedor en `authGuard`.

7.2 Funcionalidades NO Entregadas / Parciales

Dashboard con sidebar: No se implemento el layout con barra lateral de navegacion.

Estado real del perfil: Sin integracion activa con API para datos reales de la tienda.

Gestion de productos: Boton deshabilitado con etiqueta Proximamente. Fuera del alcance del Sprint I.

7.3 Metricas del Sprint

Indicador	Valor
Tareas planificadas	9
Tareas completadas	7 de 9 (78%)
Tareas parciales	2 de 9 (22%)
Horas estimadas	47 horas
Horas consumidas	21 horas

7.4 Feedback Obtenido

No se obtuvo feedback formal de stakeholders durante el Sprint I. Las revisiones se realizaron internamente entre miembros del equipo.

8. Sprint Retrospective

8.1 Que Salio Bien

- Distribucion clara de tareas con responsabilidades bien definidas.
- Stack tecnologico apropiado: Django + Angular facilito desarrollo paralelo.
- Autenticacion implementada correctamente desde el inicio con tokens e interceptor.
- Estructura del proyecto organizada por dominio permitio trabajo sin conflictos.

8.2 Que Se Puede Mejorar

- Falta de standups diarios resulto en tareas estancadas sin deteccion temprana.
- Campo owner de Store no se incluyo desde el inicio, requiriendo migracion adicional.
- No se valido rol is_seller en el dashboard inicialmente.
- Sin datos de prueba hasta el final, imposibilitando demos intermedias.
- Formulario de registro de tienda subestimado en horas.

8.3 Acciones para el Proximo Sprint

1. Implementar daily standups de 15 minutos.
2. Crear seed data al inicio del Sprint.
3. Revisar modelos contra diagramas UML antes de codificar.
4. Definir criterios de aceptacion mas claros por tarea.
5. Estimar con mas margen para tareas de integracion.

9. Conclusiones

9.1 Lecciones Aprendidas

- La comunicacion entre diseno UML e implementacion es fundamental para evitar retrabajo.
- Las tareas de integracion frontend-backend requieren mas tiempo del estimado.
- Los datos de prueba son esenciales desde el inicio del Sprint.
- La validacion de roles debe ser parte de los criterios de aceptacion desde el inicio.
- El stack Django + Angular es efectivo para desarrollo paralelo con API bien definida.

9.2 Plan para Sprint II

- Completar el layout del dashboard con sidebar de navegacion lateral.
- Integrar el dashboard con la API para datos reales de la tienda.
- Implementar la gestion de productos (CRUD) desde el dashboard.
- Desarrollar la funcionalidad de busqueda y comparacion de precios.
- Implementar daily standups y mejorar seguimiento de tareas.
- Crear y mantener seed data actualizada para pruebas y demos.

El Sprint I sentó las bases técnicas y organizacionales del proyecto NutriPrecio. A pesar de las tareas parciales, el equipo logró entregar un incremento funcional con autenticación completa, registro de tiendas y dashboard del vendedor. Las lecciones aprendidas servirán para mejorar la planificación y ejecución del Sprint II.